

Structurally Valid, Semantically Wrong: Measuring the Validation Gap for AI-Generated Web Inputs

Zahid Rahimoon

Independent Researcher

Email: TODO-HUMAN

ORCID: TODO-HUMAN

Abstract—A web application’s schema validation checks that an input is well formed, but many business rules that decide whether the input is appropriate are never enforced. We measure how often inputs that pass validation still break such rules, and whether inputs written by large language models (LLMs) are special in this respect. A Next.js testbed with an enumerated reference of 43 business rules received 3878 inputs from seven generation conditions — two LLM conditions, random, rule-based adversarial, boundary, and two reference conditions — through its real HTTP validation path, and semantic validity was decided by rule functions and an independent judge model. Of the decided, structurally valid LLM inputs, 31.6 % (95 % CI 28.9–34.4) broke at least one rule; 23.2 % did so when the model was asked only for legitimate values. The gap is not specific to AI: random structurally valid inputs broke rules more often (54.6 %), and giving the model the field’s purpose and rules lowered the odds of a violation by 78 %. Five candidate semantic validation layers were evaluated. None reached the pre-registered 70 % recall at a 5 % false-positive rate; the best fast design detected 27.1 %, mostly violations of objective rules the application could enforce directly. On a four-core CPU, the rule layer added at most 14 ms to p97.5 latency, the embedding layer 65–96 ms for a single client, the small language model about half a second and the LLM judge several seconds, the judge saturating at eight concurrent clients. The study uses one application, CPU-only open models, and a judge not yet validated against a human.

Index Terms—input validation, business logic, large language models, web applications, software testing, empirical study

I. INTRODUCTION

A web application accepts an input only after it passes validation. In the JavaScript and TypeScript ecosystem that validation is usually a schema: a declarative object stating the type, presence, length, pattern, numeric range, enumeration and format of every field, checked on the client, again on the server, and finally by database constraints. Such schemas are well understood [1], [2] and empirically studied at scale [3]. Rules that depend on the application’s purpose — that a promotional code must apply to the item being bought, that a workshop must start in the future, that a review must be about the workshop it is attached to — can sometimes be written as custom refinements, but in practice most are not written down at all. The distinction that matters is therefore between business rules an application *enforces* and the many it leaves unenforced.

That limitation is not new. Felmetzger et al. separated input-validation flaws, which admit a general specification, from logic vulnerabilities, which do not [4]. Later work detected such flaws from behavioural traces [5], client–server validation mismatches [6] and over-permissive API payloads [7]. What has changed is *who writes the input*. Large language models (LLMs) now fill forms, operate web applications on a user’s behalf, and generate test data [8]–[10].

Recent work shows that LLM output which satisfies a schema is often wrong in meaning. Schema-constrained ordering agents reach full schema validity while semantic success falls as low as a few per cent [11]; forcing a small model into a schema raised validity while executable accuracy fell in a calendar tool-call task [12]; across 21 models, schema compliance exceeds value accuracy by 15 to 25 percentage points [13]; schema-first tool interfaces reduce interface misuse while increasing schema-valid semantic misuse [14]; and in an enterprise platform the most structurally successful model had among the lowest semantic satisfaction [15]. All of this concerns model *outputs* consumed by a tool, an API or a renderer. It leaves open two questions about the input boundary of a web application, where accepted values are persisted and shown to other users or to an assistant: how often do LLM-written inputs that pass validation break the application’s business rules, and is that rate different from what other producers of structurally valid input achieve?

Web testing studies cannot answer either, because they operationalise validity as acceptance by the application [8], [9]. One such study notes that some invalid data, such as incomplete e-mail addresses, may be submitted successfully because the application lacks the corresponding check [8], and a robustness study using status codes as its oracle cannot see failures that return success [16]. Where semantic validity of generated API values is measured, a base eight-billion-parameter model produces domain-coherent values under a quarter of the time [17], but without a non-LLM comparison.

This paper measures the gap at the input boundary of one purpose-built application with an explicit rule reference, and compares LLM inputs against random, rule-based, boundary and reference inputs on the same outcome. Its contributions are:

- **C1.** A measurement of the rate at which structurally valid

inputs break unenforced business rules, for LLM and non-LLM producers alike, with the rate broken down by prompt intent, field category and target, and sensitivity analyses for each design choice that affects it.

- **C2.** A controlled comparison of seven input-generation conditions on one outcome. The comparison shows that the gap is a property of the unenforced boundary rather than of AI: random structurally valid inputs exceed LLM inputs, and field context reduces LLM violations.
- **C3.** An evaluation of five semantic validation designs — rules, embedding similarity, a small language model, a hybrid, and an LLM judge — at a bounded false-positive rate on held-out fields, showing that none reaches the pre-registered target.
- **C4.** Their request-path cost measured on commodity CPU hardware, a regime the guardrail literature we reviewed does not report [18]–[21].
- **C5.** The testbed, harness, rule reference, raw data and analysis scripts, including the defects found during execution and how each was handled.

Section II fixes terminology and Section III positions the work. Section IV states the research questions and hypotheses, frozen before data collection. Section V describes the method and Section VI the validation layers. Section VII reports results, Section VIII interprets them, and Sections IX and X give threats to validity and conclusions.

II. BACKGROUND AND TERMINOLOGY

Structural validation is every check the application performs before business logic: the runtime schema (type, presence, length, pattern, range, enumeration, format, and any refinements written into it) together with service-level checks and database constraints. An input *passes* structural validation when the application accepts it.

Semantic validity is whether a structurally valid value is appropriate for the application’s purpose: consistency across fields, domain plausibility, compliance with business rules, and safety for downstream consumers such as an LLM component. It is operationalised as satisfying every rule in a frozen, enumerated reference.

An input that passes structural validation and violates at least one rule is **SV-SI** (structurally valid, semantically invalid). The *conditional* SV-SI rate is the share of SV-SI inputs among structurally valid inputs whose rules were all decided; it is the primary outcome, because a generator that produces more well-formed values would otherwise look worse merely by passing more often.

Terminology differs across communities. The OWASP Input Validation Cheat Sheet distinguishes syntactic from semantic validation [22]; we count a range the developer wrote into the schema as structural because it is enforced. Security research speaks of logic vulnerabilities [4], [5] or business logic vulnerabilities [23]; API testing research of inter-parameter dependencies [24], [25]; one older paper uses “semantic-based user input validation” in our sense [26]. In LLM security the

same adjective often means adversarial intent [27]; our usage concerns data appropriateness, which includes that case.

III. RELATED WORK

A. Limits of structural validation

JSON Schema has a formal semantics and known complexity [1], [2], and empirical studies describe which constraints practitioners encode [3]. Inter-parameter dependencies of the form *requires*, *or*, *only-one*, *all-or-none*, *zero-or-one* and arithmetic relations cannot be stated in an OpenAPI document [24], [25], which has motivated generating server-side validation code from a separate dependency language [28]. Schema-derived fuzzers inherit the same boundary [29], and surveys of input-validation vulnerabilities remain organised around injection classes [30].

B. Logic flaws

Waler inferred likely invariants over session state and database query parameters and model-checked them, finding previously unknown logic vulnerabilities [4]. Black-box behavioural patterns [5], parameter-tampering analysis [6], mass-assignment testing [7] and excessive-data-exposure fuzzing [31] detect related classes; semantic anomaly detection in this invariant sense predates the web literature [32]. Recent work applies LLMs to find business-logic vulnerabilities in code [23], [33], surveyed among other security uses [34]. In the studies we reviewed, inputs are crafted by an attacker or tester rather than compared across producers.

C. LLM-generated inputs for forms and APIs

LLMs outperform random and type-based generators at producing inputs that applications accept: on thirty real forms, an LLM-driven approach reaches a form passing rate of 83 % against 23 % for a static type-based generator and 3 % for random crawling [9]; across 146 forms and eleven models, the best model submits over 98 % of its generated inputs [8]. The oracle in each case is acceptance by the application. Where semantic validity is measured separately, a base model reaches 22.9 % domain-coherent values against 72.4 % after fine-tuning [17]; related work mines input constraints from specifications [35], [36], studies text input for mobile interfaces [37], benchmarks form-filling agents [10], and measures schema validity of synthetic records [38]. We found no study that compares LLM inputs with random, rule-based and reference inputs on a semantic outcome.

D. Schema-valid but semantically wrong outputs

Several 2026 studies report the pattern for model outputs: OrderBench separates syntactic validity, schema validity and semantic success [11]; the constraint-tax study measures accuracy lost under hard schema decoding [12]; a multi-source benchmark finds a gap between JSON pass rate and value accuracy [13]; a tool-API study finds schemas shift failures from interface to semantic misuse [14]; and a production study finds workflows that pass validation and cannot execute [15].

Taxonomies of structured-output error [39], cross-column constraint repair [40] and neuro-symbolic verification [41] develop the theme. Tool-use benchmarks evaluate the calls an LLM emits against a task’s expected answer, under realistic multi-turn use [42] or with a focus on filling parameter values correctly [43]. In these studies the semantic check is the oracle rather than an evaluated, deployable layer, and the setting is output rather than input.

E. Data validation beyond schemas

Data-management research validates data beyond structural schemas, for example with anomaly detection on data fed to machine-learning pipelines [44], declarative constraint checks that act as unit tests for data [45], and learned detection of the semantic types of columns [46]. These systems operate on datasets or columns in batch, whereas we study per-request validation of individual inputs against an application’s business rules. Classifying a single field entry as valid or invalid from a context embedding and statistics of recent entries has been described in a patent [47]; our EMB design is a simpler, untrained relative that we evaluate empirically.

F. Semantically inappropriate input as a security problem

Indirect prompt injection established that models do not separate data from instructions [48], that attacks on deployed applications succeed [49], [50], and that surface quality checks do not distinguish compromised from clean text [51]. An attacker can insert prompt fragments through an ordinary form field that later drive a database write [52]; the pattern extends to classic web sinks [53] and occurs in the wild [54]. Benchmarks [55]–[57] and a journal study [58] extend coverage. Inputs in this line of work are attacker-crafted and mostly free text.

G. Detection layers and their cost

Input-side detectors span fine-tuned classifiers [18], [59], embedding classifiers [19], [60], small transformers [61], [62], intrinsic-feature probes [63], [64] and LLM judges [65], [66]. False-positive rate on benign traffic is the deployment constraint [67]. Reported cost evidence is thin and GPU-bound: one detector reports no timing [18]; an embedding guardrail reports a 35 ms embedding floor on a datacentre GPU [19]; a layered framework reports a 61 ms median on an A100 with model inference excluded [20]; a systematisation notes that cost and utility are routinely overlooked [21], [68], [69]. One study compares regex, classifier and LLM tiers but does not state its hardware [70], and LLM judges are themselves injectable [71]. We found no measurement of a validator for data appropriateness on structured request payloads on CPU.

IV. RESEARCH QUESTIONS AND HYPOTHESES

The following were frozen, with the primary metrics and the analysis plan, before any experimental input was generated. Prompt family P5 (injection, Section V) is excluded from all primary rates and analysed separately.

- **RQ1.** Do LLM-generated inputs pass structural validation while violating business rules? *H1*: the conditional SV-SI

rate of the LLM conditions has a 95 % Wilson interval whose lower bound exceeds 0.05.

- **RQ2.** How does this vary across field categories? *H2*: rates differ across categories; free text and cross-field highest, simple scalar and structured (enumeration) lowest.
- **RQ3.** How do LLM inputs compare with other producers? *H3a*: LLM with field context exceeds random; *H3b*: LLM with field context exceeds benign-unusual reference inputs; *H3c*: LLM without field context exceeds random.
- **RQ4.** Which violation types dominate? Exploratory.
- **RQ5.** Can an added semantic layer detect SV-SI inputs? *H5*: at least one non-judge design reaches recall ≥ 0.70 at a false-positive rate (FPR) ≤ 0.05 on held-out fields.
- **RQ6.** What does it cost? *H6*: rule and embedding designs add at most 50 ms to p95 request latency while an LLM judge adds at least 1000 ms.

V. METHOD

A. Subject under study

The subject is a purpose-built Next.js 16 application (TypeScript) using Zod 4 for validation on client and server, Prisma with SQLite for persistence, and a local Ollama runtime for two AI-facing features (support-ticket triage and a booking assistant). It has seven input surfaces and 39 fields. A reference file enumerates 43 business rules; five are enforced in code (two of them as Zod refinements: a birth date not in the future, and a workshop end after its start) and 38 are not, a profile chosen to resemble small production applications. Each rule is classified in advance as *objective* (decidable from the input, the database and the clock; 19 rules) or *judgement* (24 rules). The application and the rule reference were tagged before any input was generated (commit 62a7267), and the commit is recorded with every run. A purpose-built subject is a deliberate trade: real applications do not enumerate their business rules, and without a complete reference the semantic outcome has no ground truth.

B. Targets and generation conditions

Seventeen targets covering 22 fields were selected to represent every field category, every directly AI-facing field, and five cross-field pairs whose rule relates the members. Each target carries a purpose description, its rule identifiers and its structural constraints. Seven conditions produce values for each target: **A** valid reference values; **B** random values that satisfy the schema, drawn from a seeded generator of word sequences, numbers and dates within the schema’s limits, with no attempt at meaning; **C** rule-based adversarial values, one template per unenforced rule; **D** LLM given the field name and structural constraints only; **E** LLM additionally given the field’s purpose and rules; **F** boundary values at schema edges; **G** benign but unusual reference values. Conditions A and G were specified as human-written; in this execution their 272 values were drafted by an AI assistant at the researcher’s request, before the main generation, and checked against the

schemas and objective rules. The researcher’s review of them is outstanding (Section IX).

C. LLM generation

Conditions D and E use qwen3:4b (Q4_K_M) through Ollama on a four-core CPU, temperature 0.8, nucleus 0.9, a fixed seed per run, three runs, five values per call. Seven prompt families are stored as hashed files: P1 legitimate values, P2 unusual values, P3 values that satisfy the constraints but “would be wrong for the application to accept”, P4 values whose meaning depends on downstream interpretation, P5 injection (AI-facing targets only), P6 boundary values and P7 Unicode variants. Because P3 asks for violations, results are reported per family as well as pooled. Model identifier, digest, temperature, seed, prompt file and hash are recorded for every value. Failed calls were counted and not regenerated: 88 failure records in 52 prompt cells (75 call errors, such as calls aborted at the 120s timeout, and 13 unparseable replies). A further 430 generated values used key names other than the target’s; a harness defect had filled those keys from the form’s default values, so these records measure the defaults, not the model, and are treated as generation failures.

D. Structural measurement

Each input is measured twice: against the same schema object the application imports, and over HTTP against the real route handler, recording status, service errors and time. The structural outcome is the application’s own decision, so an input rejected by one of the enforced business checks counts as a structural rejection. Booking state is reset to the seed before each booking request.

E. Semantic ground truth

Objective rules are decided by rule functions; the 11 objective rules that apply to the 17 targets all have one. Judgement rules are decided by a judge model from a different family than the generator (gemma3:4b, temperature 0), given the field purpose, the rule text and the submitted value of the target’s fields, and never the condition, prompt family or intent. The judge does not see the rest of the record, so seven judgement rules that depend on other fields (e.g. capacity plausible for the location) were judged without that context; a sensitivity analysis excludes them. A planned human annotation of 200 inputs, which would validate the judge, has not been completed. An input is *decided* when every applicable rule has a verdict. The judge was applied to a stratified sample for compute reasons: 1200 structurally valid inputs drawn round-robin across target $\times$ condition with a fixed seed, plus 136 for two booking targets re-measured after a defect (Section IX).

F. Analysis population and statistics

Of 3878 inputs, 430 key-mismatch records, 131 records produced by inadvertent retries of failed calls, and 74 injection-family inputs are excluded, leaving 3243. Of these, 2927 passed structural validation and 1859 are decided; the remainder were not in the judge sample (1028) or received

no usable verdict (40). Rates carry Wilson 95 % intervals. H1 is evaluated on the lower bound, H2 with a χ^2 test and Cramér’s V , and H3a–c with three one-sided Fisher tests, Holm-corrected. Because the judge sample gives conditions different target mixes, each contrast is also tested within target with the Cochran–Mantel–Haenszel (CMH) test, added after data collection. Values from one call and targets within a category are not independent, so intervals are likely too narrow; per-run and per-target rates are reported for that reason. All other analyses are labelled exploratory. Every number in this paper is generated from the raw data by scripts released with it.

VI. SEMANTIC VALIDATION LAYERS

Five designs run as a drop-in check after structural validation. **R** packages the objective rule functions; since these functions also define the ground truth for objective rules, **R** cannot produce a false positive on them and serves as a floor. **EMB** scores a value by its cosine distance from an embedding (nomic-embed-text) of the field’s purpose and rules; it uses no labelled examples, unlike the trained classifier the plan anticipated. **SLM** prompts a small language model (SLM; qwen3:1.7b) with the field purpose and rules for a verdict and confidence. **HYB** runs **R**, then **EMB** on what **R** passes. **JUDGE** sends the same prompt to qwen3:4b, the generator’s model, and is a reference point; **SLM** and **JUDGE** thus receive a rubric similar to the ground-truth judge’s. Prompted designs run at temperature 0; a reply without a verdict is a pass, as the deployed layer fails open.

Evaluation is leave-fields-out over five folds. In each fold the threshold is the smallest score at which at most 5 % of valid training-fold inputs are flagged, applied to held-out fields. Designs are compared on recall at that operating point, with FPR, precision and the threshold-free area under the ROC curve (AUROC). The model-based designs were scored on a stratified sample of 579 decided inputs (236 SV-SI), on which all designs are compared; the fast designs were also scored on all 1942 decided inputs, a population that includes the injection family and retried records.

VII. RESULTS

A. Rate of semantic violations (RQ1)

Among decided, structurally valid LLM inputs, 340 of 1076 (31.6 %, 95 % CI 28.9–34.4) broke at least one business rule (Table I), so H1 is supported. This pooled rate includes prompt family P3, which asks for rule violations. Per family (Table II), the rate was 23.2 % for legitimate-value prompts and 43.3 % when violations were requested; without P3 it was 29.5 %. H1 holds under every sensitivity variant in Table III, including objective rules alone (7.9 %, CI 6.8–9.2), which do not depend on the judge. One target dominates: workshop dates (F15+F16), where nearly every model value was in the past because the prompts never state the current date; without it the rate is 22.1 %. Per run, the rate was 33.9, 41.6 and 40.9 % without field context and 27.0, 24.9 and 22.9 % with it; including the retried records gives 31.2 %.

TABLE I

STRUCTURAL AND SEMANTIC OUTCOMES BY INPUT-GENERATION CONDITION. THE CONDITIONAL SV-SI RATE IS THE SHARE OF STRUCTURALLY VALID INPUTS THAT VIOLATE AT LEAST ONE BUSINESS RULE; IT IS THE PRIMARY OUTCOME, COMPUTED OVER *decided* PASSES: THOSE WHOSE EVERY APPLICABLE RULE RECEIVED A VERDICT (PASSES OUTSIDE THE JUDGE SAMPLE ARE UNDECIDED AND EXCLUDED). PROMPT FAMILY P5 (INJECTION) AND RETRIED RECORDS ARE EXCLUDED. INTERVALS ARE WILSON 95% CONFIDENCE INTERVALS. *SPECIFIED AS HUMAN-WRITTEN; VALUES AI-DRAFTED, RESEARCHER REVIEW OUTSTANDING.

Condition	<i>n</i>	Struct. pass (%)	Passes	Decided	SV-SI	Cond. SV-SI (%)	95% CI
A valid human*	136	100.0	136	131	5	3.8	[1.6, 8.6]
B random	425	93.6	398	359	196	54.6	[49.4, 59.7]
C rule-based adversarial	85	96.5	82	82	64	78.0	[67.9, 85.6]
D LLM context-free	1268	84.0	1065	521	202	38.8	[34.7, 43.0]
E LLM context-aware	1108	92.8	1028	555	138	24.9	[21.4, 28.6]
F boundary	85	96.5	82	79	20	25.3	[17.0, 35.9]
G benign-unusual human*	136	100.0	136	132	24	18.2	[12.5, 25.6]

TABLE II

CONDITIONAL SV-SI RATE OF MODEL-GENERATED INPUTS BY PROMPT FAMILY (DECIDED STRUCTURAL PASSES; P5 EXCLUDED). P3 EXPLICITLY ASKS THE MODEL FOR VALUES THE APPLICATION SHOULD REJECT; P1 ASKS FOR LEGITIMATE VALUES. D: WITHOUT FIELD CONTEXT; E: WITH PURPOSE AND RULES.

Prompt asks for	D (%)	E (%)	SV-SI	D+E (%)	95% CI
P1 legitimate	30.3	15.8	45/194	23.2	[17.8, 29.6]
P2 unusual	36.2	26.1	52/163	31.9	[25.2, 39.4]
P3 rule violations	48.1	39.1	71/164	43.3	[35.9, 50.9]
P4 context-dependent	29.1	20.6	47/193	24.4	[18.8, 30.9]
P6 boundary	46.7	22.3	56/169	33.1	[26.5, 40.5]
P7 Unicode variation	45.6	27.2	69/193	35.8	[29.3, 42.7]
All (H1)			340/1076	31.6	[28.9, 34.4]
All except P3			269/912	29.5	[26.6, 32.5]

TABLE III

SENSITIVITY OF THE CONDITIONAL SV-SI RATE TO DESIGN CHOICES. LLM: CONDITIONS D AND E POOLED; B: RANDOM; G: BENIGN-UNUSUAL (AI-DRAFTED). “WITHOUT CONTEXT RULES” DROPS SEVEN JUDGEMENT RULES THE JUDGE HAD TO DECIDE WITHOUT THE REST OF THE RECORD. “OBJECTIVE RULES ONLY” USES NO JUDGE. TARGET-WEIGHTED AVERAGES PER-TARGET RATES BY EACH TARGET’S SHARE OF STRUCTURAL PASSES. P1 AND P3 APPLY TO MODEL CONDITIONS ONLY.

Variant	LLM (%)	95% CI	B (%)	G (%)
Pre-registered (all rules)	31.6	[28.9, 34.4]	54.6	18.2
P1 (legitimate) prompts only	23.2	[17.8, 29.6]	–	–
Without P3 (asks for violations)	29.5	[26.6, 32.5]	54.6	18.2
Without dates (F15+F16)	22.1	[19.6, 24.9]	51.8	18.5
Objective rules only	7.9	[6.8, 9.2]	10.3	0.7
Without context rules	25.0	[22.5, 27.7]	36.9	9.1
Target-weighted	30.9	–	54.7	17.9

Reference conditions bound the labelling: valid inputs were labelled invalid in 3.8% of cases and adversarial inputs in 78.0%, so about a fifth of designed violations were missed and the rates may be underestimates.

B. Field categories (RQ2)

Category is associated with the outcome ($\chi^2(6) = 179.8$, $p < 0.001$, Cramér’s $V = 0.31$, a moderate association), so H2’s test is supported (Table IV). Its directional expectation is only partly met: free text (53%) and cross-field (46%) are high as expected, but structured fields are the highest (58%)

TABLE IV

CONDITIONAL SV-SI RATE BY INPUT-FIELD CATEGORY, OVER STRUCTURALLY VALID INPUTS FROM ALL CONDITIONS. A χ^2 TEST OF INDEPENDENCE GIVES $\chi^2 = 179.8$, $p < 0.001$, Cramér’s $V = 0.311$.

Field category	Passes	SV-SI	Cond. SV-SI (%)	95% CI
Structured	107	62	57.9	[48.5, 66.9]
Free text	66	35	53.0	[41.2, 64.6]
Cross-field	597	276	46.2	[42.3, 50.2]
AI-facing	259	114	44.0	[38.1, 50.1]
Search	95	27	28.4	[20.3, 38.2]
Simple scalar	250	67	26.8	[21.7, 32.6]
Financial	485	68	14.0	[11.2, 17.4]

TABLE V

CONDITIONAL SV-SI RATE PER TARGET, ALL CONDITIONS AND MODEL CONDITIONS (D+E) ONLY, OVER DECIDED STRUCTURAL PASSES. CATEGORIES HOLD ONE TO FOUR TARGETS, SO A CATEGORY RATE CAN BE DRIVEN BY A SINGLE TARGET.

Target	Category	Decided	All (%)	LLM dec.	LLM (%)
F03	AI-facing	93	71.0	42	85.7
F35	AI-facing	81	6.2	37	2.7
F39	AI-facing	85	50.6	45	53.3
F07+F09	Cross-field	109	32.1	61	8.2
F15+F16	Cross-field	181	88.4	132	99.2
F20+F21	Cross-field	203	15.8	152	9.2
F31+F33	Cross-field	104	47.1	54	31.5
F12	Financial	105	20.0	54	20.4
F13	Financial	185	0.0	134	0.0
F14	Financial	114	39.5	63	23.8
F25	Financial	68	0.0	28	0.0
F28	Financial	13	15.4	0	–
F29	Free text	66	53.0	26	34.6
F18	Search	95	28.4	44	34.1
F01	Simple scalar	93	36.6	42	11.9
F04	Simple scalar	157	21.0	106	19.8
F36+F37	Structured	107	57.9	56	64.3

rather than among the lowest, and financial fields the lowest (14%). Categories contain one to four targets (Table V); the structured category is a single target, a ticket’s priority and category, whose rules the judge decided without seeing the ticket text, so this reversal should not be read as a property of enumerations in general.

TABLE VI

PRE-DECLARED ONE-SIDED CONTRASTS ON THE CONDITIONAL SV-SI RATE (FISHER EXACT, HOLM-CORRECTED ACROSS THE THREE TESTS). EACH TEST IS ONE-SIDED IN THE PRE-REGISTERED DIRECTION (RATE 1 > RATE 2), SO AN ODDS RATIO BELOW 1 YIELDS p CLOSE TO 1. CONDITIONS: B RANDOM, E LLM WITH FIELD CONTEXT, G BENIGN-UNUSUAL (AI-DRAFTED), D LLM WITHOUT FIELD CONTEXT. WITHIN-TARGET RESULTS ARE IN THE TEXT.

	Contrast	Rate 1 (%)	Rate 2 (%)	OR	p_{Holm}	Supported
H3a	E>B	24.9	54.6	0.28	1.00	no
H3b	E>G	24.9	18.2	1.49	0.19	no
H3c	D>B	38.8	54.6	0.53	1.00	no

TABLE VII

EXPLORATORY: SV-SI BY GENERATOR SIZE AND TEMPERATURE ON THE SAME 10 TARGETS, PROMPT FAMILIES P1–P4, CONDITIONS D AND E. THE OBJECTIVE-RULE RATE IS DECIDED FOR EVERY STRUCTURAL PASS AND SO IS COMPARABLE ACROSS ROWS; THE ALL-RULES RATE USES THE JUDGE, WHICH LABELLED 100 INPUTS PER E2B RUN AGAINST A 1,200-INPUT SAMPLE IN E1. p_{Holm} : TWO-SIDED FISHER TEST AGAINST THE E1 GENERATOR († QWEN3:4B, $T = 0.8$, THE E1 REFERENCE), HOLM-CORRECTED ACROSS THREE COMPARISONS. OBJ.: SV-SI ON OBJECTIVE RULES AMONG STRUCTURAL PASSES. ALL: SV-SI AMONG DECIDED PASSES.

qwen3	Runs	n	Pass	Obj. (%)	p_H	All (%)	p_H
1.7b, $T=0.8$	1 340	334	16 (4.8)	0.00	48/137 (35.0)	0.05	
4b, $T=0.8^\dagger$	3 878	798	9 (1.1)	ref.	74/312 (23.7)	ref.	
8b, $T=0.8$	1 394	355	5 (1.4)	0.77	30/130 (23.1)	1.00	
4b, $T=0$	1 274	252	7 (2.8)	0.15	26/99 (26.3)	1.00	

C. Comparison of producers (RQ3)

None of the three pre-registered contrasts is supported (Table VI), and two point the other way. Random structurally valid inputs broke rules in 54.6 % of cases, more often than LLM inputs without (38.8 %) or with field context (24.9 %), and this holds in every sensitivity variant (Table III). Within target (CMH, post hoc), the odds of a violation were lower for LLM inputs with context than for random inputs (odds ratio 0.16, 95 % CI 0.11–0.23) and lower without context (0.46, 0.33–0.65); adversarial inputs exceeded random ones (6.22, 2.6–14.8). Field context lowered the odds relative to the schema alone (0.22, 0.15–0.34). LLM inputs with context and benign-unusual reference inputs did not differ detectably (1.28, 0.75–2.18, $p = 0.35$); the interval is wide, and the reference inputs are AI-drafted, so this is not an AI-versus-human result. Fig. 1 shows the rates by condition.

An exploratory variation repeated conditions D and E on ten targets with families P1–P4, changing only the generator (Table VII). The 1.7-billion-parameter model produced more violations than the 4-billion model on objective rules (4.8 % against 1.1 %; odds ratio 4.4, $p_{\text{Holm}} = 0.001$) and on all rules (35.0 % against 23.7 %; $p_{\text{Holm}} = 0.05$). Neither the 8-billion model (23.1 %) nor temperature 0 (26.3 %) differed detectably. With one run per generator and a smaller judge sample, this indicates only that a smaller model may widen the gap.

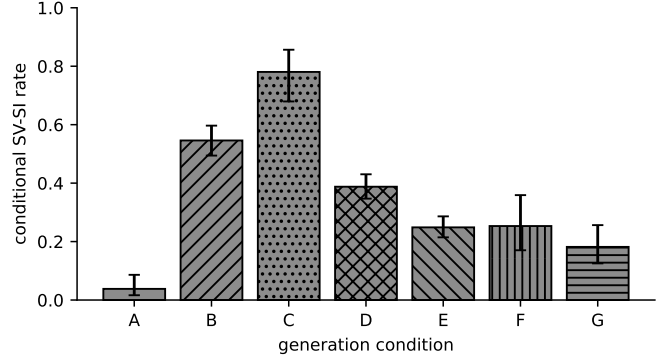


Fig. 1. Conditional SV-SI rate by generation condition with Wilson 95 % intervals, over decided structural passes (P5 excluded). C is the designed adversarial condition; A and G are the AI-drafted reference conditions for legitimate input.

D. Violation types (RQ4)

The most frequently violated rule is objective — a workshop must start in the future (146 violations) — followed by duration plausibility (57); both are rules the application could enforce in code. Among judgement rules, the most frequent were a profile bio addressed to the assistant (66), a ticket priority inconsistent with its text (62) and implausible capacity for a location (45); the last two were judged without the text or location they depend on. On AI-facing fields, 134 structurally valid inputs violated a rule; 20 came from the injection family and 114 from ordinary prompts.

E. Ground-truth agreement

Judge–human agreement is not reported, because the human annotation was not completed (TODO–HUMAN); judgement-rule results therefore rest on the judge alone. Objective-rule results, which cover every structurally valid input, do not.

F. Detection quality (RQ5)

No design reached the target, so H5 is not supported (Table VIII). At the operating point fixed on training folds, the best non-judge design, HYB, detected 27.1 % of SV-SI inputs (CI 21.8–32.7) at a held-out FPR of 5.8 %, above the 5 % it was tuned for. Its detections come from R: HYB and R caught 100.0 % of inputs violating only objective rules and 6.0 % and 0.0 % of those violating a judgement rule. On all decided inputs the fast designs reach 33.3 % (R) and 36.6 % (HYB). EMB alone was no better than chance (AUROC 0.45).

The prompted designs rank inputs better (AUROC 0.75 for SLM and 0.76 for JUDGE, against 0.60 for HYB), but their scores take only a handful of values, so holding FPR to 5 % on training fields forces the threshold to the top confidence level, where SLM detects 1.3 % and JUDGE 23.7 %. Taking each model’s own verdict without tuning (exploratory), SLM detects 69.1 % while flagging 20.4 % of valid inputs, and JUDGE 51.7 % while flagging 6.7 %; JUDGE gave no usable verdict for 25 % of inputs. Because these designs receive a rubric similar to the ground-truth judge’s, their agreement with it

TABLE VIII

DETECTION QUALITY OF THE CANDIDATE SEMANTIC VALIDATION LAYERS ON THE SHARED STRATIFIED SAMPLE OF 579 DECIDED INPUTS (236 SV-SI, 343 VALID), LEAVE-FIELDS-OUT (5 FOLDS). RECALL, 95% BOOTSTRAP INTERVAL, FPR AND PRECISION ARE AT THE PRE-REGISTERED OPERATING POINT: THE THRESHOLD IS THE SMALLEST SCORE WHOSE FALSE-POSITIVE RATE ON VALID TRAINING-FOLD INPUTS IS AT MOST 5%, APPLIED TO HELD-OUT FIELDS. AUROC IS THRESHOLD-FREE. THE NATIVE COLUMN (EXPLORATORY, NOT PRE-REGISTERED) APPLIES EACH DESIGN’S OWN DECISION RULE WITHOUT TUNING: FOR SLM AND JUDGE, THE MODEL ANSWERED “NOT APPROPRIATE”; FOR R, EMB AND HYB, SCORE ≥ 0.5 . JUDGE RETURNED NO USABLE VERDICT FOR 25% OF INPUTS, COUNTED AS NOT FLAGGED. R’S ZERO FPR IS PARTLY BY CONSTRUCTION: ITS RULE FUNCTIONS ALSO DECIDE THE GROUND TRUTH FOR OBJECTIVE RULES. JUDGE SHARES A MODEL FAMILY WITH THE GENERATOR AND IS A REFERENCE POINT. LATENCY IS THE MEDIAN DECISION TIME OF THE LAYER ALONE ON THE STUDY CPU; HYB’S FIGURE REUSES EMBEDDINGS ALREADY CACHED BY EMB, SO ITS UNCACHED REQUEST-PATH COST IS THE ONE IN TABLE IX.

Design	Recall (%)	95% CI	FPR (%)	Precision (%)	AUROC	Native recall / FPR (%)	Median (ms)
HYB	27.1	[21.8, 32.7]	5.8	76.2	0.60	24.2 / 2.3	0.01
JUDGE	23.7	[19.0, 29.1]	3.5	82.4	0.76	51.7 / 6.7	2706
R	22.5	[17.2, 27.8]	0.0	100.0	0.61	22.5 / 0.0	0.01
EMB	4.7	[2.1, 7.6]	5.8	35.5	0.45	1.7 / 2.3	31
SLM	1.3	[0.0, 3.0]	1.2	42.9	0.75	69.1 / 20.4	1075

partly measures agreement between two LLMs. Thresholds did not transfer across fields: held-out FPR per fold reached 33 % for the embedding-based designs. On benign-unusual inputs, FPR at the operating point was 6.2 % for HYB, 1.5 % for SLM and 1.5 % for JUDGE.

G. Overhead (RQ6)

Each configuration ran against the production build with only its own model loaded; no request failed and no layer call failed in any run (Table IX). H6 is not supported, because its first part fails. R added at most 1 ms to p97.5 latency with one client and 14 ms with eight, within the 50 ms bound; negative differences at eight clients reflect run-to-run variation. EMB added 65–96 ms with one client, above the bound, and 122–751 ms with eight, because every request embeds its text on the same CPU. The second part holds: SLM added 527–534 ms at the median with one client, and JUDGE 1611–7704 ms. Because the model server answers one request at a time, requests queue under concurrency: with eight clients SLM added 4149–4210 ms at the median, and JUDGE completed no request within the 60 s window in all three runs on the course endpoint and two of three on the review endpoint. Throughput on the course endpoint fell from 47.4 requests per second with structural validation alone to 1.8 with SLM and 0.1 with JUDGE. Against detection quality (Fig. 2), only R and HYB lie on the trade-off frontier; SLM, JUDGE and EMB each cost more and detect less than one of them at the pre-registered operating point.

VIII. DISCUSSION

RQ1: the gap is measurable, and its size depends on what is asked. About a third of structurally valid LLM inputs broke an unenforced rule, a quarter when the model was asked only for legitimate values, and under a tenth on objective rules alone. The rate extends output-side findings [11], [13], [15] to inputs that an application would persist, but its magnitude is sensitive to prompt intent, to one date-dependent target, and to an unvalidated judge, so the claim that survives every check is that the gap is clearly above the 5 % bound rather than any single figure.

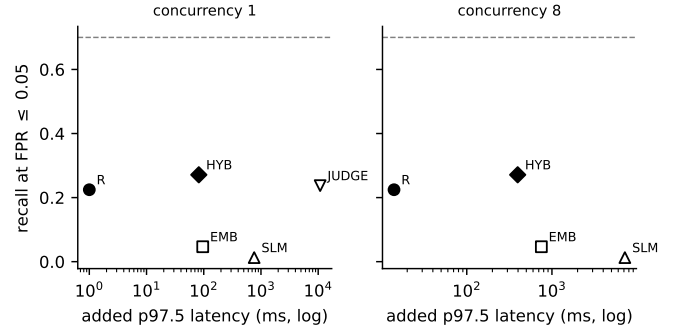


Fig. 2. Detection versus request-path cost for each semantic layer: recall at $FPR \leq 0.05$ on held-out fields (Table VIII) against the largest added p97.5 latency over the three endpoints (Table IX), per concurrency level. Filled markers are on the trade-off frontier; the dashed line is the pre-registered 70 % recall target. JUDGE at concurrency 8 saturated and is drawn at its lower bound.

RQ2: where it occurs is uneven. Category explained a moderate share of the variation, but individual targets drive it: a single target with a clock-dependent rule accounts for most of the cross-field rate, and the structured category is one target judged without its context.

RQ3: the gap is not specific to AI. This is the central negative result. Random values that satisfy the schema broke rules more often than model values, with or without field context, pooled and within target. The phenomenon is a property of the validation boundary: a schema admits many values the application should reject, and any producer that satisfies the schema without knowing the rules falls into it. What distinguishes the model is that context helps; the purpose and rules cut the odds of a violation by 78 % relative to the schema alone. The random generator produces word sequences with no attempt at meaning, so “random” here is a weak producer, and the result bounds rather than characterises non-AI input.

RQ4: most violations are ordinary. The two most frequent violations were of objective rules the application could enforce directly, and on AI-facing fields most came from ordinary prompts rather than injection attempts. The gap is mostly

TABLE IX

REQUEST-PATH OVERHEAD OF EACH CONFIGURATION (PRODUCTION BUILD, 4-CORE CPU, NO GPU). EACH CELL IS THE MEDIAN OVER 3 REPETITIONS OF A 60 S AUTOCANNON RUN AFTER A 10 S WARM-UP, WITH A UNIQUE PAYLOAD PER REQUEST. ADDED LATENCY IS RELATIVE TO STRUCTURAL-ONLY VALIDATION ON THE SAME ENDPOINT AND CONCURRENCY. H6 IS EVALUATED ON THE ADDED P97.5, THE NEAREST TAIL PERCENTILE AUTOCANNON REPORTS ($\geq$ P95). LAYER SUCCESS IS THE SHARE OF LAYER CALLS THAT RETURNED A VERDICT RATHER THAN FAILING OPEN. “>60 000”: THE CONFIGURATION SATURATED, COMPLETING NO REQUEST WITHIN AT LEAST TWO OF THE THREE 60 S RUNS.

Config.	Endpoint	p50 (ms)	p97.5 (ms)	Δ p50 (ms)	Δ p97.5 (ms)	req/s	Layer ok (%)	H6
<i>Concurrency 1</i>								
structural	courses	20	28	0	0	47.4	–	
R	courses	20	29	0	1	46.0	100	
EMB	courses	47	93	27	65	19.2	100	
HYB	courses	46	93	26	65	19.2	100	
SLM	courses	547	789	527	761	1.8	100	
JUDGE	courses	7185	10214	7165	10186	0.1	100	
structural	profile	21	32	0	0	42.9	–	
R	profile	22	33	1	1	41.9	100	
EMB	profile	57	102	36	70	15.9	100	
HYB	profile	58	100	37	68	15.6	100	
SLM	profile	552	687	531	655	1.8	100	
JUDGE	profile	1632	7715	1611	7683	0.5	100	
structural	reviews	20	31	0	0	45.6	–	
R	reviews	22	31	2	0	43.1	100	
EMB	reviews	68	127	48	96	13.2	100	
HYB	reviews	65	113	45	82	13.6	100	
SLM	reviews	554	729	534	698	1.7	100	
JUDGE	reviews	7724	10855	7704	10824	0.1	100	
<i>Concurrency 8</i>								
structural	courses	146	322	0	0	48.3	–	
R	courses	149	317	3	-5	47.6	100	
EMB	courses	272	444	126	122	27.9	100	
HYB	courses	249	453	103	131	29.0	100	
SLM	courses	4295	7178	4149	6856	1.7	100	
JUDGE	courses	>60 000	>60 000	>60 000	>60 000	0.0	100	
structural	profile	156	477	0	0	41.7	–	
R	profile	158	341	2	-136	44.8	100	
EMB	profile	355	787	199	310	20.6	100	
HYB	profile	325	752	169	275	22.9	100	
SLM	profile	4354	7257	4198	6780	1.7	100	
JUDGE	profile	14438	23951	14282	23474	0.4	100	
structural	reviews	155	333	0	0	45.6	–	
R	reviews	161	347	6	14	43.9	100	
EMB	reviews	443	1084	288	751	16.0	100	
HYB	reviews	425	729	270	396	17.7	100	
SLM	reviews	4365	7513	4210	7180	1.7	100	
JUDGE	reviews	>60 000	>60 000	>60 000	>60 000	0.0	100	

unenforced business logic.

RQ5: no cheap layer closes it. The designs that can be held to a low FPR find about a quarter to a third of violations, almost all of them objective ones that could simply be enforced. The prompted models discriminate better but give coarse, field-dependent scores; used as they are, they catch more while rejecting between one valid input in fifteen and one in five in this sample. Detecting judgement-rule violations remains the open problem. **RQ6: cost rises faster than detection.** On commodity CPU hardware only the rule layer is effectively free, and it detects violations of rules the application could have enforced in the first place. Embedding costs tens of milliseconds per request and detected almost nothing here. The prompted models cost half a second to several seconds per request with one client and, because a local model server handles one request at a time, fail to keep up with eight; their better ranking of inputs does not survive the false-positive constraint. On this hardware the practical

recommendation is to enforce objective rules in code and treat judgement rules as an open problem, rather than add a model to the request path.

Alternative explanations. The six explanations fixed in advance fare as follows. *Weak business rules rather than AI:* consistent with the data; the gap appears for every producer that does not know the rules. *Random inputs produce the same effect:* consistent, and random inputs exceed model inputs. *Unusual human inputs behave the same:* not tested, because the reference inputs are AI-drafted. *Confined to free text:* not supported; structured and cross-field targets are as high or higher, though driven by single targets. *Prompt injection under another name:* not supported; injection prompts are excluded from all rates and account for 20 of 134 violations on AI-facing fields. *Too expensive to deploy:* supported for the model-based layers on CPU hardware, not for rules.

IX. THREATS TO VALIDITY

Internal validity. Labelling is the main threat. Judgement rules were decided by one 4B judge without human validation, and seven of them without the fields they depend on; excluding those seven lowers the LLM rate to 25.0% and the random rate more (Table III), leaving the RQ3 direction intact. Agreement on semantic judgements can be low even between humans [72]. The judge sample gives conditions different target mixes; target-weighted rates are close to the pooled ones. Values generated in one call are correlated, so the intervals understate uncertainty. Time-relative rules are evaluated against the clock when labels are merged, so a later re-run can change labels near the boundary. The judge ran once; its run-to-run stability on this CPU was not measured.

Defects found and corrected. Five defects were found during execution; each is recorded as a deviation with the invalid data preserved. (i) The database was not reset between booking measurements, so course capacity ran out; bookings are now reset before each measurement and the booking targets were re-measured. (ii) A resume routine retried failed generation calls; retried records are excluded. (iii) Model outputs with other key names were silently replaced by form defaults; those records are excluded as generation failures. (iv) A first overhead run never enabled the layer and is discarded. (v) The first detector analysis selected thresholds with a quantile, which fails for coarse scores; it was replaced before any detector result was reported. Defects (iii) and (v) were found by independent review of the analysis.

External validity. One application in one domain, built by the researcher, with one enforcement profile; results may not transfer. Generation uses quantised open models of 1.7 to 8 billion parameters on CPU; frontier models may behave differently, and evidence that scale does not predict semantic quality [13], [15] cuts both ways. The prompts never state the current date, which inflates date-rule violations for model inputs.

Construct validity. Semantic invalidity is defined by a frozen reference that another team would write differently. The reference was frozen before any input existed, and violations are reported per rule so a reader can discount rules they disagree with.

Conclusion validity. Pooling three runs inflates the LLM sample; per-run rates are reported. The category mapping assigns dual-tagged fields to one category, decided before data but not logged at the time. Only the three pre-declared contrasts are confirmatory.

Provenance. The reference conditions A and G were specified as human-written and were drafted by an AI assistant; until the researcher's review is recorded, H3b is exploratory and the alternative explanation that unusual human inputs behave similarly is not tested.

X. CONCLUSION AND FUTURE WORK

Schema validation answers whether an input is well formed, not whether it belongs. At the input boundary of one application with an explicit rule reference, about a third of

structurally valid LLM inputs broke an unenforced business rule, but random structurally valid inputs broke rules more often, and telling the model the field's purpose and rules reduced violations. The gap is therefore a property of what the application leaves unenforced, not of AI; the most frequent violation was of an objective rule that a line of validation code could enforce. None of the five candidate layers detected the pre-registered share of violations at a tolerable false-positive rate. On a four-core CPU, rules added a few milliseconds, embeddings tens of milliseconds, a small language model about half a second and an LLM judge several seconds, the judge saturating under modest concurrency. Future work should complete the human-written reference conditions and the human validation of the judge, re-judge context-dependent rules with the full record, state the current date to generators, and repeat the protocol on an existing open-source application.

ARTIFACT AVAILABILITY

The testbed (tag `v1.0-frozen`), the harness, the prompts, the rule reference, every generated input, measurement and label, and the scripts that produce every table, figure and number in this paper are released with it (TODO-HUMAN: repository URL and licence).

ACKNOWLEDGMENT

The literature search, verification, drafting, harness implementation and analysis scripting were performed with the assistance of an AI system under the author's direction; the research claims and the manuscript are the author's responsibility. A full disclosure of AI use and all artefacts are released with the paper.

REFERENCES

- [1] F. Pezoa, J. L. Reutter, F. Suarez, M. Ugarte, and D. Vrgoč, "Foundations of JSON schema," in *Proceedings of the 25th International Conference on World Wide Web*, ser. WWW '16. International World Wide Web Conferences Steering Committee, Apr. 2016, pp. 263–273. [Online]. Available: <http://dx.doi.org/10.1145/2872427.2883029>
- [2] L. Atouche, M.-A. Baazizi, D. Colazzo, G. Ghelli, C. Sartiani, and S. Scherzinger, "Validation of modern JSON schema: Formalization and complexity," *Proceedings of the ACM on Programming Languages*, vol. 8, no. POPL, pp. 1451–1481, 2024. [Online]. Available: <https://doi.org/10.1145/3632891>
- [3] M.-A. Baazizi, D. Colazzo, G. Ghelli, C. Sartiani, and S. Scherzinger, *An Empirical Study on the "Usage of Not" in Real-World JSON Schema Documents*. Springer International Publishing, 2021, pp. 102–112. [Online]. Available: http://dx.doi.org/10.1007/978-3-030-89022-3_9
- [4] V. Felmetzger, L. Cavedon, C. Kruegel, and G. Vigna, "Toward automated detection of logic vulnerabilities in web applications," in *Proceedings of the 19th USENIX Security Symposium*. USENIX Association, 2010. [Online]. Available: https://www.usenix.org/legacy/event/sec10/tech/full_papers/Felmetzger.pdf
- [5] G. Pellegrino and D. Balzarotti, "Toward black-box detection of logic flaws in web applications," in *Proceedings 2014 Network and Distributed System Security Symposium*, ser. NDSS 2014. Internet Society, 2014. [Online]. Available: <http://dx.doi.org/10.14722/ndss.2014.23021>
- [6] P. Bisht, T. Hinrichs, N. Skrupsky, R. Bobrowicz, and V. N. Venkatakrishnan, "NoTamper: automatic blackbox detection of parameter tampering opportunities in web applications," in *Proceedings of the 17th ACM conference on Computer and communications security*, ser. CCS '10. ACM, Oct. 2010, pp. 607–618. [Online]. Available: <http://dx.doi.org/10.1145/1866307.1866375>

- [7] D. Corradini, M. Pasqua, and M. Ceccato, "Automated black-box testing of mass assignment vulnerabilities in RESTful APIs," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, May 2023, pp. 2553–2564. [Online]. Available: <http://dx.doi.org/10.1109/icse48619.2023.00213>
- [8] T. Li, C. Cui, R. Huang, D. Towey, and L. Ma, "Large language models for automated web-form-test generation: An empirical study," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 3, pp. 1–37, Feb. 2026. [Online]. Available: <http://dx.doi.org/10.1145/3735553>
- [9] P. Alian, N. Nashid, M. Shahbandeh, and A. Mesbah, "Semantic constraint inference for web form test generation," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA '24. ACM, Sep. 2024, pp. 932–944, arXiv:2402.00950. [Online]. Available: <http://dx.doi.org/10.1145/3650212.3680332>
- [10] B. Li, Y. Wang, H. Fei, J. Li, W. Ji, M.-L. Lee, and W. Hsu, "FormFactory: An interactive benchmarking suite for multimodal form-filling agents," in *Proceedings of the 33rd ACM International Conference on Multimedia*. ACM, 2025, pp. 13 273–13 280. [Online]. Available: <https://doi.org/10.1145/3746027.3758285>
- [11] Y. Li, "When JSON is not enough: Semantic reliability of schema-constrained LLM ordering agents," arXiv preprint arXiv:2607.18261, 2026. [Online]. Available: <https://arxiv.org/abs/2607.18261>
- [12] J. Ray, "The constraint tax: Measuring validity-correctness tradeoffs in structured outputs for small language models," arXiv preprint arXiv:2605.26128, 2026. [Online]. Available: <https://arxiv.org/abs/2605.26128>
- [13] A. K. Singh, H. V. Khurdula, Y. D. Khemlani, and V. Agarwal, "The structured output benchmark: A multi-source benchmark for evaluating structured output quality in large language models," arXiv preprint arXiv:2604.25359, 2026. [Online]. Available: <https://arxiv.org/abs/2604.25359>
- [14] A. Sigdel and R. Baral, "Schema first tool APIs for LLM agents: A controlled study of tool misuse, recovery, and budgeted performance," arXiv preprint arXiv:2603.13404, 2026. [Online]. Available: <https://arxiv.org/abs/2603.13404>
- [15] X. Wrenn, R. Raykov, A. Angelov, H. Kitahara, Y. Watanabe, and A. Sailer, "Evaluating LLM trade-offs for enterprise automation: Lessons from workflow generation in a production enterprise platform," arXiv preprint arXiv:2608.03311, 2026. [Online]. Available: <https://arxiv.org/abs/2608.03311>
- [16] H. G. Tigulla and M. Vieira, "LLM-based robustness testing of microservice applications: An empirical study," arXiv preprint arXiv:2605.14202, 2026. [Online]. Available: <https://arxiv.org/abs/2605.14202>
- [17] M. Kim, S. Sinha, and A. Orso, "LlamaRestTest: Effective REST API testing with small language models," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 465–488, Jun. 2025. [Online]. Available: <http://dx.doi.org/10.1145/3715737>
- [18] D. Jacob, H. Alzahrani, Z. Hu, B. Alomair, and D. Wagner, "PromptShield: Deployable detection for prompt injection attacks," in *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy*, ser. CODASPY '25. ACM, Jun. 2024, pp. 341–352, arXiv:2501.15145. [Online]. Available: <http://dx.doi.org/10.1145/3714393.3726501>
- [19] I. Ahmed, A. A. Borsha, R. D. Prangon, A. fuad Ahmad, and T. H. Tran, "Reflex-guard: A low-latency guardrail for LLM prompt safety using dense semantic embeddings," arXiv preprint arXiv:2608.17556, 2026. [Online]. Available: <https://arxiv.org/abs/2608.17556>
- [20] G. Saleem, N. Ahmed, M. I. Zaman, A. Hassan, and U. Mujahid, "A layered security framework against prompt injection in RAG-based chatbots," arXiv preprint arXiv:2606.19660, 2026. [Online]. Available: <https://arxiv.org/abs/2606.19660>
- [21] X. Wang, Z. Ji, W. Wang, Z. Li, D. Wu, and S. Wang, "SoK: Evaluating jailbreak guardrails for large language models," in *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2026, pp. 39–58. [Online]. Available: <https://doi.org/10.1109/SP63933.2026.00076>
- [22] OWASP Cheat Sheet Series Team, "Input validation cheat sheet," OWASP Cheat Sheet Series, OWASP Foundation, 2026, accessed: Sep; 22, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
- [23] B. Metin, M. Wynn, A. Tunali, and Y. Kepir, "Business logic vulnerabilities in the digital era: A detection framework using artificial intelligence," *Information*, vol. 16, no. 7, p. 585, Jul. 2025. [Online]. Available: <http://dx.doi.org/10.3390/info16070585>
- [24] A. Martin-Lopez, S. Segura, C. Muller, and A. Ruiz-Cortes, "Specification and automated analysis of inter-parameter dependencies in web APIs," *IEEE Transactions on Services Computing*, vol. 15, no. 4, pp. 2342–2355, Jul. 2022. [Online]. Available: <http://dx.doi.org/10.1109/tsc.2021.3050610>
- [25] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés, *A Catalogue of Inter-parameter Dependencies in RESTful Web APIs*. Springer International Publishing, 2019, pp. 399–414. [Online]. Available: http://dx.doi.org/10.1007/978-3-030-33702-5_31
- [26] S. Hanna and M. Munro, "Test case generation for semantic-based user input validation of web applications," *International Journal of Web Engineering and Technology*, vol. 13, no. 3, p. 225, 2018. [Online]. Available: <http://dx.doi.org/10.1504/ijwet.2018.095186>
- [27] N. Singh, A. Abuadbbba, Y. Gao, S. Nepal, and H. Kim, "Shifting from injection to interaction: Rethinking web security in the age of LLMs and beyond," arXiv preprint arXiv:2609.03999, 2026. [Online]. Available: <https://arxiv.org/abs/2609.03999>
- [28] S. Barakat, A. B. Sánchez, and S. Segura, *IDLGen: Automated Code Generation for Inter-parameter Dependencies in Web APIs*. Springer Nature Switzerland, 2023, pp. 153–168. [Online]. Available: http://dx.doi.org/10.1007/978-3-031-48421-6_11
- [29] Z. Hatfield-Dodds and D. Dygalo, "Deriving semantics-aware fuzzers from web API schemas," arXiv preprint arXiv:2112.10328, 2021. [Online]. Available: <https://arxiv.org/abs/2112.10328>
- [30] F. Faisal Fadlalla and H. T. Elshoush, "Input validation vulnerabilities in web applications: Systematic review, classification, and analysis of the current state-of-the-art," *IEEE Access*, vol. 11, pp. 40 128–40 161, 2023. [Online]. Available: <http://dx.doi.org/10.1109/access.2023.3266385>
- [31] L. Pan, S. Cohnney, T. Murray, and V.-T. Pham, "EDEFuzz: A web API fuzzer for excessive data exposures," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. ACM, Feb. 2024, pp. 1–12. [Online]. Available: <http://dx.doi.org/10.1145/3597503.3608133>
- [32] O. Raz, P. Koopman, and M. Shaw, "Semantic anomaly detection in online data sources," in *Proceedings of the 24th international conference on Software engineering - ICSE '02*, ser. ICSE '02. ACM Press, 2002, p. 302. [Online]. Available: <http://dx.doi.org/10.1145/581376.581378>
- [33] M. Armillotta, N. Romandini, R. Montanari, and L. Cavallaro, "Antaeus: Hunting repository-level logic vulnerabilities via context-grounded LLM reasoning," arXiv preprint arXiv:2607.01138, 2026. [Online]. Available: <https://arxiv.org/abs/2607.01138>
- [34] Z. Sheng, Z. Chen, S. Gu, H. Huang, G. Gu, and J. Huang, "LLMs in software security: A survey of vulnerability detection techniques and insights," *ACM Computing Surveys*, vol. 58, no. 5, pp. 1–35, Nov. 2025. [Online]. Available: <http://dx.doi.org/10.1145/3769082>
- [35] M. Kim, T. Stennett, D. Shah, S. Sinha, and A. Orso, "Leveraging large language models to improve REST API testing," in *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, ser. ICSE-NIER'24. ACM, Apr. 2024, pp. 37–41. [Online]. Available: <http://dx.doi.org/10.1145/3639476.3639769>
- [36] H. Huynh, Q.-T. Le, T. N. Nguyen, and V. Nguyen, "Using LLM for mining and testing constraints in API testing," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24. ACM, Oct. 2024, pp. 2486–2487. [Online]. Available: <http://dx.doi.org/10.1145/3691620.3695341>
- [37] C. Cui, T. Li, J. Wang, C. Chen, D. Towey, and R. Huang, "Large language models for mobile gui text input generation: An empirical study," arXiv preprint arXiv:2404.08948, 2024. [Online]. Available: <https://arxiv.org/abs/2404.08948>
- [38] S. Mishra, "Reliability-aware structured synthetic data generation via schema enforcement and layered repair," in *Proceedings of the Workshop on Synthetic Data Generation and Management for Building AI Systems*, ser. SynthAI '26. ACM, May 2026, pp. 7–12. [Online]. Available: <http://dx.doi.org/10.1145/3814574.3816747>
- [39] Y. Song, P. Rajput, T. Sun, S. Ezzini, T. F. Bissyandé, and J. Klein, "Empirical study for structured output control in LLMs for software engineering," arXiv preprint arXiv:2606.09395, 2026. [Online]. Available: <https://arxiv.org/abs/2606.09395>
- [40] J. Zhao, M. Guan, and D. Liu, "Constraint-aware synthetic tabular data generation via inter-column constraint discovery with LLM

- agents,” arXiv preprint arXiv:2608.15109, 2026. [Online]. Available: <https://arxiv.org/abs/2608.15109>
- [41] P. Sigloch and C. Benzmler, “Neuro-symbolic verification of LLM outputs for data-sensitive domains (extended preprint),” arXiv preprint arXiv:2605.26942, 2026. [Online]. Available: <https://arxiv.org/abs/2605.26942>
- [42] P. Yu, W. Liu, Y. Yang, J. Li, Z. Zhang, X. Feng, and F. Zhang, “Benchmarking LLM tool-use in the wild,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2026, arXiv:2604.06185. [Online]. Available: <https://openreview.net/forum?id=yz7fL5vfpn>
- [43] G. Yu, X. Sun, Z. Huang, S. Fan, Z. Zhang, X. Hu, X. Xue, Y. Shi, X. Xiao, Y. Song, B. Lyu, R. Wen, X. Li, Q. He, S. Zhu, and Z. Liu, “Getting the parameters right: A difficulty-graded benchmark and probe-guided training for LLM tool calls,” arXiv preprint arXiv:2608.03071, 2026. [Online]. Available: <https://arxiv.org/abs/2608.03071>
- [44] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, “Data validation for machine learning,” in *Proc. Machine Learning and Systems (MLSys)*, vol. 1, 2019, pp. 334–347. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2019/hash/928f1160e52192e3e0017fb63ab65391-Abstract.html
- [45] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, “Automating large-scale data quality verification,” *Proc. VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018.
- [46] M. Hulsebos, K. Hu, M. Bakker, E. Zraggen, A. Satyanarayan, T. Kraska, C. Demiralp, and C. Hidalgo, “Sherlock: A deep learning approach to semantic data type detection,” in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2019, pp. 1500–1508.
- [47] A. Banerjee and V. T. Subrahmaniam, “Deep learning based context embedding approach for detecting data entry errors,” US Patent 12288159, Apr. 29, 2025. [Online]. Available: <https://patents.google.com/patent/US12288159B2>
- [48] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, ser. CCS ’23. ACM, Nov. 2023, pp. 79–90, arXiv:2302.12173. [Online]. Available: <http://dx.doi.org/10.1145/3605764.3623985>
- [49] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, L. Y. Zhang, and Y. Liu, “Prompt injection attack against LLM-integrated applications,” arXiv preprint arXiv:2306.05499, 2023. [Online]. Available: <https://arxiv.org/abs/2306.05499>
- [50] Y. Kaya, A. Landerer, S. Pletinckx, M. Zimmermann, C. Kruegel, and G. Vigna, “When AI meets the web: Prompt injection risks in third-party AI chatbot plugins,” in *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2026, pp. 4223–4242. [Online]. Available: <https://doi.org/10.1109/SP63933.2026.00062>
- [51] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *Proceedings of the 33rd USENIX Security Symposium*. USENIX Association, 2024, pp. 1831–1847. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei>
- [52] R. Pedro, M. E. Coimbra, D. Castro, P. Carreira, and N. Santos, “Prompt-to-SQL injections in LLM-integrated web applications: Risks and defenses,” in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, Apr. 2025, pp. 1768–1780, arXiv:2308.01990. [Online]. Available: <http://dx.doi.org/10.1109/icse55347.2025.00007>
- [53] S. Tsigkopoulos and C. Ntantogian, “From prompt injection to web exploitation: Revisiting classic vulnerabilities in LLM-integrated applications,” arXiv preprint arXiv:2608.10281, 2026. [Online]. Available: <https://arxiv.org/abs/2608.10281>
- [54] S. Khodayari, X. Zhang, B. Acharya, and G. Pellegrino, “Indirect prompt injection in the wild: An empirical study of prevalence, techniques, and objectives,” arXiv preprint arXiv:2604.27202, 2026. [Online]. Available: <https://arxiv.org/abs/2604.27202>
- [55] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, “Benchmarking and defending against indirect prompt injection attacks on large language models,” in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*, ser. KDD ’25. ACM, Jul. 2025, pp. 1809–1820. [Online]. Available: <http://dx.doi.org/10.1145/3690624.3709179>
- [56] I. Evtimov, A. Zharmagambetov, A. Grattafiori, C. Guo, and K. Chaudhuri, “WASP: Benchmarking web agent security against prompt injection attacks,” in *Advances in Neural Information Processing Systems* 38. Neural Information Processing Systems Foundation, Inc., 2025, pp. 22436–22458. [Online]. Available: <https://doi.org/10.52202/085713-0666>
- [57] Y. Liu, R. Xu, X. Wang, Y. Jia, and N. Z. Gong, “WAIInjectBench: Benchmarking prompt injection detections for web agents,” arXiv preprint arXiv:2510.01354, 2025. [Online]. Available: <https://arxiv.org/abs/2510.01354>
- [58] A. Milani, V. Franzoni, and E. Florindi, “Indirect prompt injection in large language models,” *Neural Computing and Applications*, vol. 38, no. 13, Jun. 2026. [Online]. Available: <http://dx.doi.org/10.1007/s00521-026-12266-x>
- [59] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, “DataSentinel: A game-theoretic detection of prompt injection attacks,” in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, May 2025, pp. 2190–2208. [Online]. Available: <http://dx.doi.org/10.1109/sp61157.2025.00250>
- [60] M. A. Ayub and S. Majumdar, “Embedding-based classifiers can detect prompt injection attacks,” in *Proceedings of the Conference on Applied Machine Learning in Information Security (CAMLIS 2024)*, ser. CEUR Workshop Proceedings, R. Allen, S. Samtani, E. Raff, and E. Rudd, Eds., vol. 3920. Arlington, Virginia, USA: CEUR-WS.org, 2024, pp. 257–268. [Online]. Available: <https://ceur-ws.org/Vol-3920/paper15.pdf>
- [61] Y. Datta and S. Rajasekar, “JavelinGuard: Low-cost transformer architectures for LLM security,” arXiv preprint arXiv:2506.07330, 2025. [Online]. Available: <https://arxiv.org/abs/2506.07330>
- [62] K. Lakara, R. Shi, and F. Silavong, “Fence: Specialized SLM guardrails for LLM applications,” arXiv preprint arXiv:2607.18268, 2026. [Online]. Available: <https://arxiv.org/abs/2607.18268>
- [63] T. Shi, K. Zhu, Z. Wang, Y. Jia, W. Cai, W. Liang, H. Wang, H. Alzahrani, J. Lu, K. Kawaguchi, B. Alomair, X. Zhao, W. Y. Wang, N. Gong, W. Guo, and D. Song, “PromptArmor: Simple yet effective prompt injection defenses,” arXiv preprint arXiv:2507.15219, 2025. [Online]. Available: <https://arxiv.org/abs/2507.15219>
- [64] A. Akinrele and S. N. Gowda, “Prompt injection detection is regime-dependent: A deployment-aware evaluation with interpretable structural signals,” arXiv preprint arXiv:2605.26999, 2026. [Online]. Available: <https://arxiv.org/abs/2605.26999>
- [65] H. X. Le, B. Goh, and Q. A. Tang, “Prompt attack detection with LLM-as-a-judge and mixture-of-models,” arXiv preprint arXiv:2603.25176, 2026. [Online]. Available: <https://arxiv.org/abs/2603.25176>
- [66] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, “NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Y. Feng and E. Lefever, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 431–445. [Online]. Available: <https://aclanthology.org/2023.emnlp-demo.40/>
- [67] H. Li and X. Liu, “InjecGuard: Benchmarking and mitigating over-defense in prompt injection guardrail models,” arXiv preprint arXiv:2410.22770, 2024. [Online]. Available: <https://arxiv.org/abs/2410.22770>
- [68] G. Zizzo, G. Cornacchia, K. Fraser, M. Z. Hameed, A. Rawat, B. Buesser, M. Purcell, P.-Y. Chen, P. Sattigeri, and K. Varshney, “Adversarial prompt evaluation: Systematic benchmarking of guardrails against prompt input attacks on LLMs,” arXiv preprint arXiv:2502.15427, 2025, presented at the NeurIPS 2024 Safe Generative AI Workshop (non-archival). [Online]. Available: <https://arxiv.org/abs/2502.15427>
- [69] D. Kumar, N. A. Birur, T. Baswa, S. Agarwal, and P. Harshangi, “No free lunch with guardrails,” arXiv preprint arXiv:2504.00441, 2025. [Online]. Available: <https://arxiv.org/abs/2504.00441>
- [70] A. C. Maiorano, “Evaluating prompt injection defenses for educational LLM tutors: Security-usability-latency trade-offs,” arXiv preprint arXiv:2605.06669, 2026. [Online]. Available: <https://arxiv.org/abs/2605.06669>
- [71] J. Shi, Z. Yuan, Y. Liu, Y. Huang, P. Zhou, L. Sun, and N. Z. Gong, “Optimization-based prompt injection attack to LLM-as-a-judge,” in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’24. ACM, 2024, pp. 660–674. [Online]. Available: <https://doi.org/10.1145/3658644.3690291>
- [72] T. Calò, A.-E. Gurita, and L. De Russis, “Measuring the semantic accessibility gap in LLM-generated web UIs,” in *Proceedings of the Extended Abstracts of the 2026 CHI Conference on Human Factors*

in Computing Systems, ser. CHI EA '26. ACM, Apr. 2026, pp. 1–5.
[Online]. Available: <http://dx.doi.org/10.1145/3772363.3799364>